1. Aufgabe: Versionierung/Versionsverwaltung (5 Punkte)

- Was versteht man im Zusammenhang mit Versionsverwaltung unter Checkout?
    - Beim Checkout erzeugt ein Entwickler nach dem sogenannten Sandbox-Prinzip zunächst eine lokale Arbeitskopie des Repository-Inhalts, auf die sich alle Programmänderungen vorerst beschränken. Ob dieser Vorgang jederzeit möglich ist, hängt von der Locking-Strategie ab: Beim Optimistic Locking ist das Auschecken immer erlaubt, während es beim Pessimistic Locking nur möglich ist, wenn kein anderer Entwickler die Datei bereits ausgecheckt hat.

2. Aufgabe: Design Patterns (40 Punkte)

```
public interface Observable {
    public void addObserver(PersonenObserver obs);
    public void removeObserver(PersonenObserver obs);
    public void notifyObservers();
}


public interface PersonenObserver {
    public void update();
}


public class PersonenModel implements Observable{
    private Vector<String> personen = new Vector<>();
    private Set<PersonenObserver> observers = new HashSet<>();
        ...
    @Override
    public void addObserver(PersonenObserver obs) {
        observers.add(obs);
    }

    @Override
    public void removeObserver(PersonenObserver obs) {
        observers.remove(obs);
    }

    @Override
    public void notifyObservers() {
        for(PersonenObserver observer: observers){
            observer.update();
        }
    }
}
```

```
public class PersonenControl{
    private PersonenModel personenModel;
    private PersonenView personenView;

    public PersonenControl() {
        this.personenModel = new PersonenModel();
        this.personenView = new PersonenView(this.personenModel, this);
        // Test
        this.personenModel.lesePersonenAusDatei();
        this.personenModel.notifyObservers();
        // this.personenView.updateAnzeigePersonen();
    }
}


public class PersonenView implements PersonenObserver{
    PersonenModel personenModel;
    PersonenControl personenControl;

    public PersonenView(PersonenModel personenModel,
                        PersonenControl personenControl) {
        this.personenModel = personenModel;
        this.personenControl = personenControl;

        this.personenModel.addObserver(this);
    }

    public void update(){
        System.out.println(this.personenModel.gibPersonenAus());
    }
}
```

3. Aufgabe: Vertiefung Java (30 Punkte)

a)

```
public class Person {
    private String vorname;
    private String nachname;
    private String familienstand;

    public Person(String vorname, String nachname, String familienstand) {
        this.vorname = vorname;
        this.nachname = nachname;
        this.familienstand = familienstand;
    }

    public boolean istLedig() {
        return familienstand.equals("ledig");
    }

    public String getNachname() {
        return nachname;
    }
    public void setNachname(String nachname) {
        this.nachname = nachname;
    }
    public String getVorname() {
        return vorname;
    }
    public void setVorname(String vorname) {
        this.vorname = vorname;
    }
    public String getFamilienstand() {
        return familienstand;
    }
    public void setFamilienstand(String familienstand) {
        this.familienstand = familienstand;
    }
}


public class Anwendersystem {
    public static void main() {
        ArrayList<Person> personen = new ArrayList<>();
        personen.add(new Person("Alex", "Müller", "verheiratet"));
        personen.add(new Person("Kevin", "Schröder", "ledig"));

        for(Person person: personen){
            System.out.println(person.getVorname() + " " +
                person.getNachname() + ": " + person.istLedig());
        }
    }
}
```

b)

```
public class Anwendersystem {
    public static void main() {
        Person person = new Person("Musterfrau", "Elke", "ledig");
        Supplier<String> stringSupplier = () -> "Ledig? ";
        BooleanSupplier booleanSupplier = () -> person.istLedig();

        System.out.println(stringSupplier.get() +
                        booleanSupplier.getAsBoolean());

    }
}
```

Das Konzept der Pure Function ist hier nicht erfüllt:

- **Externer Zustand**: Der Lambda-Ausdruck greift auf das äußere, veränderliche Person-Objekt zu.
- **Nicht deterministisch**: Ändert sich das Objekt im Programmverlauf (z. B. durch einen Setter), liefert der Supplier ein anderes Ergebnis.
- **Verletzung der Regeln**: Eine Pure Function darf aber keine externen Abhängigkeiten haben und muss bei gleichen Parametern stets exakt dasselbe Ergebnis liefern

4. Aufgabe: JUnit 5 und Zustandsbasierter Test (25 Punkte)

JUnit 5

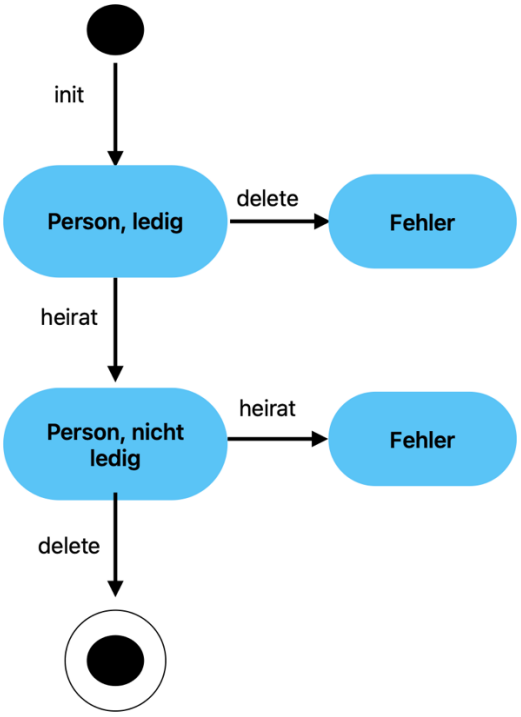
```
public class PersonTest {
    private Person person;

    @BeforeEach
    public void setUp() throws Exception {
        this.person = new Person("Mustermann", "Max", "ledig");
    }

    @AfterEach
    public void tearDown() throws Exception {
        this.person = null;
    }

    @Test
    public void test() throws Exception {
        assertTrue(
                () -> person.istLedig() == true,
                () -> "Die Methode 'istLedig()' ist ungültig!"
        );
    }
}
```

Zustandsbasierter Test



Testfälle:
init / heirat / delete
→ Es wurde ein Objekt Person (ledig) angelegt, geheiratet und dann gelöscht

init / delete
→ Es wurde ein Objekt Person (ledig) angelegt und dann gelöscht. Fehlermeldung beim Löschen

5. Aufgabe: SOLID und Clean Code (20 Punkte)

a) Single Responsibility Principle mit Beispiel:

- **Prinzip**: Eine Klasse darf nur genau eine einzige Aufgabe (Verantwortlichkeit) haben.
- **Beispiel**: Die Klasse Person verwaltet nur die reinen Personendaten (Name, Familienstand). Sie darf nicht dafür zuständig sein, diese Daten in einer Datenbank zu speichern oder sie für eine Webseite zu formatieren.

b) Warum hilft das bei Wartbarkeit und Fehlern?

- **Trennung**: Daten und Speicher-Logik sind voneinander getrennt.
- **Weniger Fehler**: Wenn das Datenbanksystem ausgetauscht wird, bleibt die Klasse Person völlig unangetastet. Das verhindert, dass man aus Versehen Fehler in der eigentlichen Personen-Logik einbaut (sogenannte Seiteneffekte).

c) Clean Code Wert "Wandelbarkeit" und Beispiel:

- **Wert**: Software muss so gebaut sein, dass sie sich leicht an neue Anforderungen anpassen lässt.
- **Beispiel**: Weil die Klasse Person von der Speicher-Logik befreit ist, können wir später völlig problemlos neue Speicherwege (z. B. Speichern in einer Cloud) ergänzen, ohne den bestehenden Person-Code umbauen zu müssen.

d) Zusammenhang von SOLID, Clean Code und Graden:

- **Zusammenhang**: Clean Code beschreibt die Ziele (wie Wandelbarkeit oder Lesbarkeit). Die SOLID-Prinzipien sind die konkreten Werkzeuge, um diese Ziele beim Programmieren zu erreichen.
- **Grade**: Um Entwicklern das Lernen zu erleichtern, werden diese Prinzipien in verschiedene Grade (Lernstufen, z. B. roter oder oranger Grad) eingeteilt. Man arbeitet sich Schritt für Schritt durch diese Grade nach oben.